

# GeCode + Pascal Bridge

## Reglas y construcción de la clase Space

Referencia técnica para el paradigma dirigido por JSON

---

### 1. Introducción

En GeCode, todo problema de satisfacción de restricciones (CSP) se modela como una clase C++ que hereda de `Gecode::Space`. Este es el contrato fundamental del framework: el programador define el problema, GeCode se encarga de la búsqueda.

En el paradigma tradicional, esta clase se escribe a mano para cada problema. En el paradigma del Bridge Pascal/JSON, la clase se construye dinámicamente en tiempo de ejecución a partir de un archivo JSON, eliminando la necesidad de recompilar.

### 2. Esquema general de la clase

Una clase GeCode válida tiene la siguiente estructura mínima obligatoria:

```
class MiProblema : public Gecode::Space {
protected:
    // BLOQUE 1: Variables de decisión
    Gecode::IntVarArray x;
    Gecode::FloatVarArray xf;

public:
    // BLOQUE 2: Constructor principal
    MiProblema() : x(*this, N, min, max) {
        // restricciones
        // estrategia de ramificación (branch)
    }

    // BLOQUE 3: Constructor de copia (OBLIGATORIO)
    MiProblema(MiProblema& s) : Space(s) {
        x.update(*this, s.x);
    }
}
```

```

// BLOQUE 4: Método copy() (OBLIGATORIO)
virtual Space* copy() {
    return new MiProblema(*this);
}

// BLOQUE 5: Método print() (recomendado)
void print() const { ... }
};

```

### 3. Reglas de construcción

La siguiente tabla resume las reglas y su nivel de obligatoriedad:

| #  | Regla                          | Descripción                                                                                                                        | Estado      |
|----|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------|-------------|
| R1 | <b>Hereda de Gecode::Space</b> | La clase DEBE heredar públicamente de Gecode::Space. Sin esto el motor de búsqueda no puede gestionar el espacio.                  | OBLIGATORIO |
| R2 | <b>Variables como miembros</b> | Las variables (IntVar, FloatVar, BoolVar, etc.) deben ser miembros de la clase. No pueden ser locales al constructor.              | OBLIGATORIO |
| R3 | <b>Constructor principal</b>   | Debe inicializar las variables con *this como primer argumento, agregar todas las restricciones y definir la estrategia branch().  | OBLIGATORIO |
| R4 | <b>Constructor de copia</b>    | Firma exacta: MiClase(MiClase& s) : Space(s). Debe llamar a update(*this, s.var) sobre cada variable miembro.                      | OBLIGATORIO |
| R5 | <b>Método copy()</b>           | Debe retornar new MiClase(*this). GeCode lo llama para clonar el espacio durante la búsqueda y el backtracking.                    | OBLIGATORIO |
| R6 | <b>branch() en constructor</b> | Sin branch(), el solver no sabe cómo explorar. Debe especificarse qué variables ramificar y con qué heurística.                    | OBLIGATORIO |
| R7 | <b>Método print()</b>          | No obligatorio pero prácticamente siempre presente. Permite inspeccionar los valores de las variables en cada solución encontrada. | OPCIONAL    |
| R8 | <b>Método constrain()</b>      | Solo necesario para optimización (Branch and Bound). Permite imponer que la próxima solución sea mejor que la actual.              | OPCIONAL    |

## 4. Descripción detallada de cada bloque

### 4.1 Bloque 1 — Variables de decisión

Las variables son el corazón del modelo CSP. Representan los valores que el solver debe encontrar. Deben declararse como miembros de la clase para que GeCode pueda actualizarlas durante la copia del espacio.

```
IntVar    → variable entera (más común)
FloatVar  → variable de punto flotante (CPFloat en el bridge)
BoolVar   → variable booleana (0 o 1)
IntVarArray → arreglo de variables enteras
FloatVarArray → arreglo de variables flotantes
```

```
protected:
    Gecode::IntVarArray vars;      // arreglo de enteros
    Gecode::FloatVarArray fvars;  // arreglo de flotantes
    Gecode::BoolVar flag;         // variable booleana individual
```

**REGLA CRÍTICA:** Las variables nunca se declaran como locales dentro del constructor. Si son locales, GeCode no puede copiarlas correctamente y el solver fallará silenciosamente.

### 4.2 Bloque 2 — Constructor principal

El constructor principal tiene tres responsabilidades fijas que deben ejecutarse en orden:

```
MiProblema(int n, int minVal, int maxVal)
: vars(*this, n, minVal, maxVal)  // 1) Inicializar variables
{
    // 2) Agregar restricciones
    Gecode::distinct(*this, vars);
    Gecode::rel(*this, vars[0], IRT_GQ, vars[1]);

    // 3) Definir ramificación (branch)
    Gecode::branch(*this, vars, INT_VAR_SIZE_MIN(), INT_VAL_MIN());
}
```

| Paso                     | Detalle                                                                                                                                                    |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Inicializar variables | Se hace en la lista de inicialización del constructor. El primer argumento siempre es *this. Luego se especifican cantidad, mínimo y máximo del dominio.   |
| 2. Agregar restricciones | Se llaman las funciones de restricción de GeCode: rel(), distinct(), linear(), etc. Cada una recibe *this como primer argumento para asociarse al espacio. |
| 3. Definir branch()      | Especifica qué variables ramificar y con qué heurística de selección                                                                                       |

|  |                                                            |
|--|------------------------------------------------------------|
|  | de variable y valor. Sin branch() el DFS no puede avanzar. |
|--|------------------------------------------------------------|

### 4.3 Bloque 3 — Constructor de copia

Es el bloque más crítico y el que más errores genera cuando se implementa incorrectamente. GeCode lo invoca automáticamente durante el backtracking, cada vez que necesita clonar el estado del espacio.

```
// Firma EXACTA – no puede cambiar
MiProblema(MiProblema& s) : Space(s) {
    // update() actualiza cada variable con su contraparte del espacio original
    vars.update(*this, s.vars);
    fvars.update(*this, s.fvars);
    flag.update(*this, s.flag);
}
```

- ✗ Olvidar llamar a Space(s) en la lista de inicialización
- ✗ Olvidar actualizar alguna variable miembro con update()
- ✗ Usar el operador de asignación = en lugar de update()
- ✗ Cambiar la firma (usar const& o añadir parámetros extra)

### 4.4 Bloque 4 — Método copy()

Es la interfaz que usa el motor de búsqueda para obtener copias del espacio. Siempre tiene la misma implementación de una línea.

```
virtual Gecode::Space* copy() {
    return new MiProblema(*this); // llama al constructor de copia
}
```

**Nota:** En versiones modernas de GeCode ( $\geq 6.x$ ) el método es `copy()` sin parámetros. En versiones anteriores recibía un `bool share`. Verificar la versión instalada.

---

## 5. Estrategias de ramificación (branch)

La función `branch()` determina cómo explora el espacio de búsqueda el motor DFS. Una mala elección puede hacer que el solver sea exponencialmente más lento.

### 5.1 Para variables enteras (IntVar)

```
// Selección de variable – cuál asignar primero
INT_VAR_SIZE_MIN() → la de dominio más pequeño (recomendada)
```

```

INT_VAR_NONE()      → en orden de declaración
INT_VAR_RND(seed)   → aleatoria

// Selección de valor – qué valor asignar
INT_VAL_MIN()       → el mínimo del dominio (recomendado)
INT_VAL_MAX()       → el máximo
INT_VAL_MED()       → el valor medio
INT_VAL_RND(seed)   → aleatorio

```

## 5.2 Para variables flotantes (FloatVar / CPFloat)

```

// Selección de variable
FLOAT_VAR_SIZE_MIN() → dominio de menor tamaño
FLOAT_VAR_NONE()     → en orden de declaración

// Selección de valor (siempre bisección)
FLOAT_VAL_SPLIT_MIN() → mitad inferior del dominio
FLOAT_VAL_SPLIT_MAX() → mitad superior del dominio

```

---

## 6. Motores de búsqueda

El motor de búsqueda se instancia en el main (o en el bridge) recibiendo el espacio inicial. GeCode ofrece varios motores según el tipo de problema.

| Motor      | Clase           | Uso                                                                       |
|------------|-----------------|---------------------------------------------------------------------------|
| <b>DFS</b> | DFS<MiProblema> | Busca todas las soluciones. El más común.                                 |
| <b>BAB</b> | BAB<MiProblema> | Optimización (Branch and Bound). Requiere constrain().                    |
| <b>LDS</b> | LDS<MiProblema> | Limited Discrepancy Search. Útil cuando branch es buena pero no perfecta. |

---

## 7. Cómo el Bridge JSON reemplaza la clase manual

En el paradigma tradicional, el programador escribe la clase para cada problema. En el Bridge Pascal/GeCode, la clase se construye en tiempo de ejecución a partir del JSON. El mapeo es directo:

| Elemento en el JSON                 | Equivale a en la clase C++                              |
|-------------------------------------|---------------------------------------------------------|
| variables: [{name, min, max, type}] | Declaración de miembros + inicialización en constructor |

|                                 |                                                                |
|---------------------------------|----------------------------------------------------------------|
| constraints: [{type, vars, op}] | Llamadas a rel(), distinct(), linear(), etc. en el constructor |
| Llamada a csp_solve_all()       | El main con DFS<>motor.next() iterando soluciones              |
| type: "float" en variable       | FloatVar miembro + FLOAT_VAR_SIZE_MIN() en branch              |

---

## 8. Checklist antes de usar la clase

- ✓ La clase hereda de Gecode::Space (R1)
- ✓ Todas las variables son miembros, no locales al constructor (R2)
- ✓ El constructor inicializa variables con \*this en la lista de inicialización (R3)
- ✓ El constructor agrega todas las restricciones con rel/distinct/linear/etc (R3)
- ✓ El constructor termina con branch() (R6)
- ✓ El constructor de copia llama a Space(s) y actualiza todas las variables con update() (R4)
- ✓ El método copy() retorna new MiClase(\*this) (R5)
- ✓ El motor de búsqueda (DFS/BAB) recibe el espacio como puntero (R3 del main)
- ✓ Se hace delete del espacio inicial y de cada solución retornada por next()

— Fin del documento —